

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Xây dựng hệ thống giao đồ ăn YummyZoom

HOÀNG NGUYỄN VŨ

vu.hn215171@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: TS. Đỗ Tuấn Anh _____

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 01/2026

MỤC LỤC

DANH MỤC HÌNH VẼ

CHƯƠNG 1. Giới thiệu và mục tiêu triển khai

1.1 Mục tiêu

Mục tiêu của báo cáo này là mô tả rõ hệ thống đang chạy được cấu hình và triển khai như thế nào, từ quá trình đóng gói dịch vụ Web API dưới dạng container, cơ chế triển khai trên Azure Container Apps, đến cách quản lý bí mật cấu hình và quan sát (observability) của hệ thống.

1.2 Phạm vi

Báo cáo tập trung vào backend và hạ tầng Azure (Web API, Redis, PostgreSQL, Key Vault, Log Analytics, ACR, .NET Aspire Dashboard), đồng thời mô tả ngắn gọn quy trình CI/CD thông qua GitHub Actions và công cụ Azure Developer CLI (azd). Các thành phần ứng dụng di động và cổng web quản trị chỉ được nhắc đến ở mức liên quan luồng tương tác với backend.

```
[../DoAn.tex]subfiles
```

```
listings xcolor
```

```
codegreenrgb0,0.6,0 codegrayrgb0.5,0.5,0.5 codepurplergb0.58,0,0.82 backcolour-  
rgb0.95,0.95,0.92
```

```
mystyle backgroundcolor=backcolour, commentstyle=codegreen, keywordstyle=magenta,  
numberstyle=codegray, stringstyle=codepurple, basicstyle=, breakatwhitespace=false, breaklines=true,  
captionpos=b, keepspaces=true, numbers=left, numbersep=5pt, showspaces=false,  
showstringspaces=false, showtabs=false, tabsize=2
```

```
style=mystyle
```

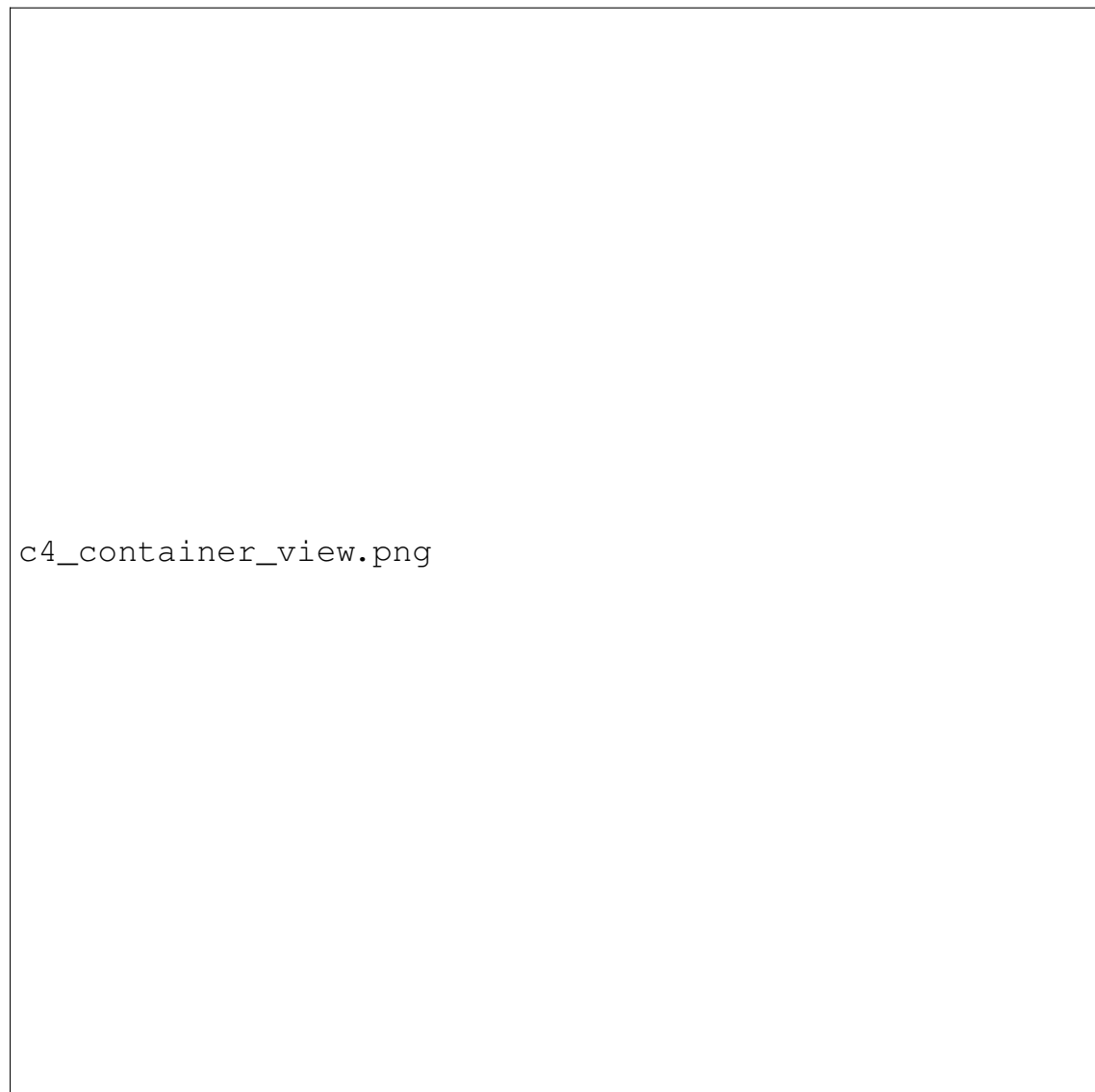
```
pythonstyle language=Python, style=mystyle
```

CHƯƠNG 2. Kiến trúc triển khai tổng quan

Chương này trình bày kiến trúc tổng thể của hệ thống YummyZoom thông qua hai góc nhìn: tổ chức các container phần mềm (Logic) và mô hình triển khai thực tế trên hạ tầng đám mây Azure (Vật lý).

2.1 Kiến trúc Container và Phân rã Logic

Hệ thống được thiết kế theo mô hình Clean Architecture, lấy Web API làm trung tâm điều phối và tách biệt rõ ràng các tầng trách nhiệm (Hình ??).



Hình 2.1: Sơ đồ kiến trúc hệ thống (C4 Container View)

Như thể hiện trong Hình ??, kiến trúc vận hành dựa trên sự phối hợp chặt chẽ giữa bốn nhóm thành phần chính, tạo thành một luồng xử lý khép kín từ người dùng đến dữ liệu và các dịch vụ vệ tinh.

Tại điểm khởi đầu, các Client Apps đóng vai trò là cổng giao tiếp chính. Người dùng cuối truy cập hệ thống thông qua ứng dụng di động đa nền tảng được xây dựng bằng Flutter, trong khi đội ngũ quản trị vận hành hệ thống qua Web Portal phát triển trên Angular. Mọi tương tác của người dùng từ việc lướt danh sách món ăn đến đặt hàng đều được chuyển tải về máy chủ thông qua giao thức HTTPS bảo mật cho các tác vụ Request-Response tiêu chuẩn, hoặc qua kênh WebSocket (SignalR) đối với các tính năng đòi hỏi sự tức thời như theo dõi vị trí tài xế hay nhận thông báo đơn hàng.

Tiếp nhận các yêu cầu này là Khối Backend Core, cốt lõi của hệ thống chạy trên nền tảng .NET 9 ASP.NET Core. Tại đây, các module nghiệp vụ (Business Logic) thực thi các quy tắc cốt lõi của miền ứng dụng Food Delivery, được bảo vệ bởi lớp xác thực tập trung (Identity) nhằm đảm bảo chỉ những yêu cầu hợp lệ mới được xử lý. Bên cạnh đó, Realtime Hub chịu trách nhiệm duy trì hàng nghìn kết nối thời gian thực, đóng vai trò như tháp điều không tin cậy điều phối luồng thông tin giữa các bên tham gia.

Dữ liệu sau khi xử lý được lưu trữ bền vững tại Tầng Dữ liệu và Tích hợp (Data & Integration). PostgreSQL 16 với phần mở rộng PostGIS được sử dụng làm kho lưu trữ chính (Primary Store) cho các dữ liệu quan trọng và phức tạp như thông tin địa lý, đơn hàng và danh mục. Song song đó, Redis đóng vai trò là bộ nhớ đệm (Cache) giúp giảm tải cho cơ sở dữ liệu chính và hoạt động như một trục liên lạc (Pub/Sub Bus) để đồng bộ trạng thái giữa các phiên bản ứng dụng khác nhau.

Cuối cùng, để tối ưu hóa nguồn lực và tập trung vào nghiệp vụ cốt lõi, hệ thống tích hợp sâu với Nhóm Dịch vụ bên thứ ba (External SaaS). Quy trình nghiệp vụ thường xuyên gọi đến các dịch vụ này: khi người dùng tìm kiếm quán ăn, Mapbox cung cấp dữ liệu bản đồ số; khi hình ảnh món ăn được tải lên, Cloudinary đảm nhận việc lưu trữ và tối ưu hóa media; khi thanh toán đơn hàng, giao dịch được xử lý an toàn qua Stripe; và khi trạng thái đơn hàng thay đổi, FCM (Firebase Cloud Messaging) sẽ lập tức gửi thông báo đẩy đến thiết bị người dùng, khép lại vòng đời của một luồng nghiệp vụ.

2.2 Kiến trúc triển khai Vật lý trên Azure

Mô hình triển khai thực tế trên Azure (Hình ??) tận dụng tối đa các dịch vụ PaaS và Serverless để giảm thiểu gánh nặng quản trị hạ tầng.



Hình 2.2: Sơ đồ triển khai vật lý trên Microsoft Azure

Quan sát Hình ??, luồng triển khai và vận hành được liên kết chặt chẽ thông qua các thành phần hạ tầng cốt lõi. Trung tâm của hệ thống là Azure Container Apps Environment, nơi lưu trữ và vận hành ba thành phần chính: Web API Service (.NET 9) đóng vai trò xử lý yêu cầu, Redis Component cung cấp bộ nhớ đệm nội bộ qua giao thức TCP, và Aspire Dashboard tiếp nhận các tín hiệu OTel (OpenTelemetry) để trực quan hóa trạng thái sức khỏe của ứng dụng.

Về mặt tích hợp dữ liệu và bảo mật, Web API kết nối tới PostgreSQL Flexible Server thông qua giao thức TCP/SSL để truy xuất dữ liệu bền vững. Các thông tin nhạy cảm (Secrets) được quản lý tập trung tại Key Vault và chỉ được truy cập thông qua cơ chế Managed Identity, đảm bảo nguyên tắc không lưu trữ mật khẩu trong mã nguồn. Tương tự, Managed Identity cũng cấp quyền để Container App kéo (Pull) các Image mới nhất từ ACR Registry.

Toàn bộ quy trình trên được điều phối tự động bởi GitHub Actions. Pipeline

CI/CD thực hiện song song hai nhiệm vụ: đóng gói và đẩy Container Image lên ACR, đồng thời sử dụng Bicep (thông qua lệnh ‘azd up’) để cung cấp tài nguyên hạ tầng. Cuối cùng, mọi dữ liệu nhật ký và số liệu giám sát từ môi trường chạy đều được đẩy về Log Analytics Workspace, tạo thành một vòng lặp DevSecOps khép kín và an toàn. Chi tiết về quy trình pipeline sẽ được trình bày trong Chương ??.

2.3 Phân tích và so sánh kiến trúc

Các quyết định công nghệ quan trọng được tóm tắt trong Bảng ??, cân bằng giữa hiệu quả chi phí và khả năng mở rộng.

Bảng 2.1: Phân tích quyết định kiến trúc (ADR)

Hạng mục	Lựa chọn	Lý do chính
Compute	Container Apps	Serverless, không cần quản lý Kubernetes Cluster phức tạp.
Database	Postgres Flexible	Tối ưu cho dữ liệu địa lý (PostGIS), chi phí thấp hơn SQL Server.
IaC	Azure Bicep	Cú pháp đơn giản, tích hợp sâu vào quy trình Azure CLI.

Đáng chú ý trong lựa chọn công nghệ IaC, đề án ưu tiên sử dụng Azure Bicep thay vì Terraform. Mặc dù Terraform có lợi thế về đa nền tảng, Bicep được chọn nhờ khả năng tích hợp native với Azure, loại bỏ gánh nặng quản lý tệp trạng thái (state file) phức tạp và tận dụng tối đa sự hỗ trợ từ công cụ Azure Developer CLI (azd) để đơn giản hóa quy trình triển khai.

So với mô hình máy ảo (VM) truyền thống, kiến trúc này loại bỏ hoàn toàn việc cấu hình thủ công OS/Runtime và vá lỗi bảo mật (patching). Thời gian khởi tạo tài nguyên giảm từ vài phút xuống vài giây, và việc cập nhật ứng dụng diễn ra không gián đoạn (Zero Downtime) nhờ cơ chế Revisions của Container Apps.

Kết chương

Chương này đã giới thiệu kiến trúc tổng thể dưới góc nhìn logic và vật lý, làm nền tảng cho các nội dung chi tiết về mã hóa hạ tầng (IaC) sẽ được trình bày trong chương kế tiếp.

CHƯƠNG 3. Hạ tầng dưới dạng mã (IaC): azd + Bicep + Azure Container Apps

Chương này đi sâu vào phương pháp "Hạ tầng dưới dạng mã" (Infrastructure as Code - IaC) mà đồ án áp dụng để định nghĩa và quản lý tài nguyên trên Azure. Thay vì thao tác thủ công trên giao diện Azure Portal là cách làm dễ gây lỗi và khó tái lập, đồ án sử dụng bộ công cụ hiện đại gồm .NET Aspire, Bicep và Azure Developer CLI (azd) để tự động hóa hoàn toàn quy trình này.

3.1 Vai trò của .NET Aspire AppHost trong định nghĩa hạ tầng

Trong kiến trúc truyền thống, mã nguồn ứng dụng (C#) và mã nguồn hạ tầng (Terraform/YAML) thường bị tách biệt, dẫn đến sự thiếu đồng bộ. .NET Aspire giải quyết vấn đề này bằng cách đưa định nghĩa hạ tầng vào ngay trong code C# thông qua dự án AppHost.

Về mặt nguyên lý, AppHost đóng vai trò là "Nguồn chân lý" (Source of Truth) cho kiến trúc logic. Nó định nghĩa các thành phần của hệ thống và mối quan hệ giữa chúng. Tại đây, đồ án tận dụng tính năng "mô hình hóa tài nguyên" của Aspire để tạo ra sự tương đồng giữa môi trường phát triển (Dev) và sản phẩm (Production).

Ví dụ, đối với dịch vụ Redis:

- **Môi trường Dev:** Khi chạy cục bộ, lệnh `AddRedis("redis")` sẽ tự động kéo Docker Image của Redis về và chạy một container tạm thời.
- **Môi trường Production:** Khi triển khai, Aspire hiểu rằng tài nguyên này ánh xạ tới một dịch vụ Redis thực thụ trên Azure. Cụ thể trong đồ án này, nó được ánh xạ tới một Azure Container App chạy Redis image thông qua phương thức `AddConnectionString("redis")`.

Đoạn mã sau trong `Program.cs` minh họa logic phân tách môi trường này:

```
[language=[Sharp]C, caption=Cấu hình Dev/Prod Parity trong AppHost, label=code:apphost-parity] var isPublishMode = builder.ExecutionContext.IsPublishMode;
```

```
if (isPublishMode) // Production: Su dung Azure Database for PostgreSQL var postgres = builder.AddAzurePostgresFlexibleServer("postgres"); else // Development: Su dung Local Container var postgres = builder.AddPostgres("postgres").WithImage("postgis/postgis", "16-3.4") // PostGIS support .WithPgAdmin();
```

Cách tiếp cận này mang lại lợi ích kép: vừa giúp lập trình viên kiểm soát kiến trúc ngay tại thời điểm biên dịch (Compile-time check), vừa đảm bảo môi trường Dev phản ánh chính xác cấu trúc của hệ thống thật.

3.2 Kiến trúc Bicep Modules và Tài nguyên Azure

Đồ án sử dụng ngôn ngữ Bicep để mô tả chi tiết các tài nguyên vật lý trên Azure. Các file Bicep được tổ chức thành các module nhỏ gọn, nằm trong thư mục `src/AppHost/infra/`, giúp dễ dàng quản lý và tái sử dụng.

3.2.1 Tài nguyên nền tảng (Shared Resources)

Module `resources.bicep` chịu trách nhiệm khởi tạo các thành phần nền tảng dùng chung cho toàn bộ hệ thống:

- **Azure Container Apps Environment:** Môi trường Serverless quản lý các container, được cấu hình với Profile `Consumption` để tối ưu chi phí theo mức sử dụng thực tế.
- **Log Analytics Workspace:** Kho lưu trữ trung tâm cho tất cả nhật ký (Logs) và chỉ số (Metrics) của hệ thống.
- **Azure Container Registry (ACR):** Kho lưu trữ riêng tư cho các Docker Image của dự án.
- **User Assigned Managed Identity:** Một danh tính định danh duy nhất dùng chung cho các tác vụ nội bộ như kéo Image từ ACR hay truy cập bí mật từ Key Vault.

3.2.2 Tài nguyên dữ liệu (Data Resources)

Các tài nguyên lưu trữ được định nghĩa riêng biệt để đảm bảo an toàn dữ liệu:

- **PostgreSQL Flexible Server:** Sử dụng SKU `Standard_B1ms` (Burstable) phù hợp cho tải biến thiên. Đáng chú ý, module này tự động kích hoạt Extension `postgis` để hỗ trợ các truy vấn không gian địa lý.
- **Redis Component:** Thay vì sử dụng dịch vụ Azure Redis Cache đắt đỏ, đồ án triển khai Redis dưới dạng một Container App riêng biệt trong cùng Environment, giao tiếp qua mạng nội bộ, giúp tiết kiệm chi phí đáng kể cho giai đoạn MVP.

Cấu hình Bicep dưới đây ('`postgres.module.bicep`') cho thấy cách tài nguyên được định nghĩa chi tiết, bao gồm cả phiên bản và các tiện ích mở rộng:

```
[language=Python, caption=Định nghĩa tài nguyên PostgreSQL với Bicep, label=code:bicep-  
postgres] resource postgres 'Microsoft.DBforPostgreSQL/flexibleServers@2024-  
08-01' = name: take('postgres-uniqueString(resourceGroup().id)', 63) sku :  
name : 'Standard_B1ms' // Cost – optimized SKU tier : 'Burstable' properties :  
version : '16' storage : storageSizeGB : 32
```

```
resource postgresExtensions 'Microsoft.DBforPostgreSQL/flexibleServers/configurations@2
```

08-01' = name: 'azure.extensions' parent: postgres properties: value: 'postgres,pg_trgm,unaccent'

3.2.3 Aspire Dashboard

Thành phần Dashboard giám sát được triển khai như một .NET Component ngay trong môi trường Container Apps. Nó tiếp nhận các tín hiệu OTel (Logs, Traces, Metrics) từ Web API, cung cấp giao diện trực quan để theo dõi sức khỏe hệ thống theo thời gian thực.

3.3 Quy trình cấp phát với Azure Developer CLI (azd)

Công cụ `azd` đóng vai trò là "nhạc trưởng" điều phối toàn bộ quy trình cấp phát và triển khai. File cấu hình `azure.yaml` định nghĩa dự án và liên kết code với hạ tầng.

Quy trình triển khai tiêu chuẩn diễn ra qua các bước:

1. **azd provision:** Lệnh này phân tích các file Bicep, so sánh trạng thái mong muốn với thực tế trên Azure, và thực hiện cấp phát hoặc cập nhật tài nguyên.
2. **State Management:** `azd` quản lý trạng thái môi trường (như Resource ID) trong thư mục `.azure/`. Các biến môi trường nhạy cảm được lưu trong file `.env` và tự động liên kết (binding) vào tham số của Bicep khi chạy lệnh.

3.4 Chiến lược Quản lý Bảo mật và Cấu hình

Đồ án áp dụng triệt để mô hình bảo mật Zero Trust thông qua việc sử dụng ****Managed Identity****.

Thay vì lưu trữ thông tin nhạy cảm như Mật khẩu CSDL hay Chuỗi kết nối (Connection String) trong mã nguồn (Hardcode)—một lỗ hổng bảo mật phổ biến, đồ án sử dụng ****Azure Key Vault****.

1. Module `keyvault.module.bicep` tạo Key Vault và thiết lập quyền truy cập (RBAC) chặt chẽ.
2. Chuỗi kết nối tới PostgreSQL và Redis được sinh ra động trong quá trình Provision và lưu trực tiếp vào Key Vault thông qua module `keyvault-secrets.module.bicep`.
3. Khi Web API khởi động, nó sử dụng Managed Identity để xác thực và đọc các cấu hình này từ Key Vault một cách an toàn.

Cơ chế này đảm bảo rằng ngay cả đội ngũ phát triển cũng không nhất thiết phải biết các thông tin nhạy cảm của môi trường Production, giảm thiểu rủi ro rò rỉ dữ liệu.

Kết chương

Chương này đã trình bày chi tiết cách thức hiện thực hóa kiến trúc hệ thống thông qua mã nguồn, từ việc định nghĩa logic trong AppHost đến cấp phát tài nguyên vật lý bằng Bicep và azd. Việc áp dụng các chuẩn mực bảo mật như Managed Identity và Key Vault cũng được phân tích cụ thể. Chương tiếp theo sẽ mô tả quy trình tích hợp và triển khai liên tục (CI/CD) để tự động hóa vòng đời phát triển này.

CHƯƠNG 4. Quy trình phát triển và Vận hành (DevOps & CI/CD)

Chương này trình bày quy trình tích hợp và triển khai liên tục, tập trung vào vòng đời container image và cơ chế phát hành revision trên Azure Container Apps.

4.1 Luồng build và push container image

Nội dung tạm (skeleton): Mô tả pipeline build/test, docker build, docker push lên ACR, tagging/versioning theo commit hoặc release.

4.2 Luồng deploy trên Azure Container Apps

Nội dung tạm (skeleton): Nhấn mạnh Azure Container Apps không build image; thay vào đó tạo revision mới trỏ tới image tag/digest, pull image từ ACR và chạy container. Mô tả traffic shifting/rollout (nếu có).

Kết chương

Chương này đã mô tả quy trình CI/CD từ build đến deploy thông qua ACR và Azure Container Apps. Chương tiếp theo trình bày các khía cạnh vận hành và quan sát hệ thống.

CHƯƠNG 5. Vận hành, quan sát hệ thống và mở rộng

Chương này tập trung vào observability (logs/metrics/traces), cấu hình theo dõi vận hành và các nguyên tắc mở rộng (scaling) phù hợp cho hệ thống MVP.

5.1 Observability: OpenTelemetry, Aspire Dashboard và Log Analytics

Nội dung tạm (skeleton): Mô tả OTel signals, cách hiển thị trên Aspire Dashboard, cơ chế log ingestion/diagnostic settings về Log Analytics, và lợi ích khi gỡ lỗi/đánh giá hiệu năng.

5.2 Quản lý bí mật và cấu hình: Key Vault + Managed Identity

Nội dung tạm (skeleton): Mô tả cách tách secrets khỏi mã nguồn/pipeline, và cơ chế MI truy cập Key Vault.

5.3 Scaling và các trade-off (zero-scale, cold start)

Nội dung tạm (skeleton): Mô tả scaling của Web API trên ACA, Redis min-Replicas=0 và trade-off cold start, cùng định hướng tối ưu chi phí.

Kết chương

Chương này đã trình bày cách hệ thống được quan sát và vận hành trong môi trường triển khai, cùng các nguyên tắc mở rộng phù hợp. Phần kết luận sau đây tổng kết các điểm chính và đề xuất hướng phát triển.

Kết luận

Báo cáo này đã trình bày kiến trúc triển khai và quy trình triển khai hạ tầng cho dự án YummyZoom trên Azure, nhấn mạnh việc sử dụng hạ tầng dưới dạng mã, CI/CD và observability để đảm bảo tính tái lập và khả năng vận hành. Trong tương lai, hệ thống có thể được hoàn thiện thêm về khả năng mở rộng realtime (ví dụ dịch vụ SignalR quản lý), cơ chế cảnh báo (alerting) và tối ưu hoá chi phí vận hành.